

Instituto Tecnológico y de Estudios Superiores de Monterrey

Desarrollo de aplicaciones avanzadas de ciencias computacionales Gpo 505

Elda Guadalupe Quiroga González

Babyduck Entrega #4

Gabriel Eduardo Diaz Roa A00833574

22 de abril del 2025

1. Expresiones Regulares

Elemento léxico	Token	Expresión Regular
id	ID	[a-zA-Z_][a-zA-Z0-9_]*
Números Enteros	INT	[0-9]+
Números Flotantes	FLOAT	[0-9]+\.[0-9]+
Strings	STRING	\".*?\"
+	SUM	\+
-	MINUS	\-
*	MULTI	*
/	DIV	\/
<	L_THAN	<
>	G_THAN	>
!=	N_EQUAL	!=
=	ASSIGN	=
;	SEMI_COL	;
:	COLON	:
,	COMMA	,
(	L_PAREN	(
)	R_PAREN	)
{	L_BRACE	{
}	R_BRACE	}
[	L_BRACKET	[
]	R_BRACKET	]
program	PROGRAM	program
main	MAIN	main
end	END	end
var	VAR	var
print	PRINT	print
if	IF	if
else	ELSE	else
while	WHILE	While
do	DO	do

2. Tokens

Palabras clave:

program, main, end, print, if, else, while, do, var

Identificadores y constantes:

id, int, float, string

Operadores:

Aritméticos: +, -, *, /

Relacionales: <, >, !=

Asignación: =

Símbolos y signos de puntuación

Semicolon ;, Colon :, Comma ,

Paréntesis: L_PAREN (, R_PAREN)

Llaves: L_BRACE {, R_BRACE }

Corchetes: L_BRACKET [, R_BRACKET]

3. Context Free Grammar

programa - 'program' ID ';' vars funcs 'main' body 'end'

vars - var_decl vars

vars - ϵ

var_decl - 'var' tipo ID mas_ids ';'

mas_ids - ';' ID mas_ids

mas_ids - ϵ

tipo - 'int'

tipo - 'float'

funcs - function funcs

funcs - ϵ

function - tipo ID '(' params_opt ')' ':' vars body ';'

params_opt - params

params_opt - ϵ

params - tipo ID mas_params

mas_params - ';' tipo ID mas_params

mas_params - ϵ

body - '{' statements '}'

statements - statement statements

statements - ϵ

statement - assign ';'

statement - condition

statement - cycle

statement - print

assign - ID '=' expresion

print - 'print' '(' expresion mas_expresiones ')' ';'

mas_expresiones - ';' expresion mas_expresiones

mas_expresiones - ϵ

condition - 'if' '(' expresion ')' body else_opt

else_opt - 'else' body

else_opt - ϵ

cycle - 'while' '(' expresion ')' 'do' body

expresion - exp expresion_opcional

expresion_opcional - relop exp

expresion_opcional - ϵ

relop - '=='

relop - '!='

relop - '<'

relop - '<='

relop - '>'

relop - '>='

exp - termino exp_suma

exp_suma - '+' termino exp_suma
exp_suma - '-' termino exp_suma
exp_suma - ϵ

termino - factor exp_mult

exp_mult - '*' factor exp_mult
exp_mult - '/' factor exp_mult
exp_mult - ϵ

factor - '(' expresion ')'
factor - CTE_INT
factor - CTE_FLOAT

factor - CTE_STRING
factor - ID
factor - f_call

f_call - ID '(' args_opt ')'

args_opt - args
args_opt - ϵ

args - expresion mas_args

mas_args - ',' expresion mas_args
mas_args - ϵ

Análisis de Herramientas de Generación de Compiladores

Herramientas Investigadas:

1. Flex & Bison
2. ANTLR
3. JavaCC

1. Flex & Bison

- Son herramientas clásicas del entorno Unix/Linux, herederas de Lex & Yacc.
- Flex genera analizadores léxicos (scanners) basados en autómatas finitos deterministas (DFA).
- Bison genera analizadores sintácticos (parsers) basados en técnicas LALR(1).
- Están diseñadas para funcionar en lenguaje C, lo cual puede dificultar su integración con otros lenguajes modernos como Java.
- Utilizan la línea de comandos como interfaz principal.
- Son muy estables y ampliamente usadas en sistemas de tipo UNIX.
- Licencia: GPL.

2. ANTLR (Another Tool for Language Recognition)

- Herramienta moderna escrita en Java.
- Permite generar analizadores léxicos y sintácticos para múltiples lenguajes de destino como Java, Python, C#, JavaScript y Go.
- Combina la definición de lexer y parser en un solo archivo .g4, facilitando la organización de proyectos.
- Utiliza técnicas de análisis LL(*) predictivo, permitiendo gramáticas más naturales sin necesidad de eliminar recursión izquierda.
- Genera automáticamente componentes adicionales como listeners y visitors para recorrer árboles de sintaxis.
- Ampliamente utilizada en la industria y en el ámbito académico para diseñar lenguajes, interpretes y compiladores.
- Licencia: BSD.

3. JavaCC (Java Compiler Compiler)

- Generador de analizadores léxicos y sintácticos escrito completamente en Java.
- Produce código Java puro sin dependencias externas en tiempo de ejecución.
- Utiliza técnicas de análisis LL(k) recursivo descendente.
- Es más sencillo de integrar en proyectos Java pequeños o medianos que no requieran árboles de sintaxis complejos.

- Comparado con ANTLR, su popularidad ha disminuido, aunque sigue siendo útil para ciertos tipos de proyectos.
- Licencia: BSD.

Conclusión del análisis

ANTLR resulta la mejor opción para el desarrollo de Baby_Duck, debido a su perfecta integración con Java, su robustez, su facilidad de uso y su documentación actualizada.

Definición de la Gramática de BabyDuck

Componente léxico	Token	Expresión regular
Identificador	ID	[a-zA-Z_][a-zA-Z_0-9]*
Constante entera	CTE_INT	[0-9]+
Constante flotante	CTE_FLOAT	[0-9]+ '.' [0-9]+
Cadena de texto	CTE_STRING	"" (~["\\"])* ""
Palabras clave	KEYWORDS	program, main, end, var, int, float, void, print, if, else, while, do
Operadores	OPERATORS	+, -, *, /, ==, !=, <, <=, >, >=
Delimitadores	SYMBOLS	;, :, ,, (,), {, }, [,]

```
badito — -zsh — 39x42

PROGRAM      : 'program' ;
MAIN         : 'main' ;
END          : 'end' ;
VAR          : 'var' ;
INT          : 'int' ;
FLOAT        : 'float' ;
VOID         : 'void' ;
PRINT        : 'print' ;
IF           : 'if' ;
ELSE         : 'else' ;
WHILE        : 'while' ;
DO           : 'do' ;

PLUS         : '+' ;
MINUS        : '-' ;
MULT         : '*' ;
DIV          : '/' ;
ASSIGN       : '=' ;
EQ           : '==' ;
NEQ          : '!=' ;
LT           : '<' ;
LEQ          : '<=' ;
GT           : '>' ;
GEQ          : '>=' ;

SEMICOLON    : ';' ;
COLON        : ':' ;
COMMA        : ',' ;
LPAREN       : '(' ;
RPAREN       : ')' ;
LBRACE       : '{' ;
RBRACE       : '}' ;
LBRACKET     : '[' ;
RBRACKET     : ']' ;

ID           : [a-zA-Z_][a-zA-Z_0-9]* ;
CTE_INT      : [0-9]+ ;
CTE_FLOAT    : [0-9]+ '.' [0-9]+ ;
CTE_STRING   : '"' (~["\\])* '"' ;

WS           : [ \t\r\n]+ -> skip ;
LINE_COMMENT : '//' ~[\r\n]* -> skip ;
```

Cada regla gramatical sigue la estructura:

- Nombre de la regla
- Secuencia de tokens y subreglas
- Uso de repeticiones (*), opciones (?) y agrupaciones

```
grammar BabyDuck;

@header {
package lex;
}

options {
    language = Java;
}

// -----
//  PARSE RULES
// -----
programa      : PROGRAM ID SEMICOLON vars funcs MAIN body END ;

vars          : var_decl* ;

var_decl      : VAR tipo ID (COMMA ID)* SEMICOLON ;

tipo          : INT
              | FLOAT ;

funcs         : funcion* ;

funcion       : tipo ID LPAREN parametros? RPAREN COLON vars body SEMICOLON ;

parametros    : tipo ID (COMMA tipo ID)* ;

body          : LBACE statements RBACE ;

statements    : statement* ;

statement     : assign SEMICOLON
              | condition
              | cycle
              | print
              | f_call SEMICOLON ;

assign        : ID ASSIGN expresion ;

print         : PRINT LPAREN (expresion | CTE_STRING) ( COMMA (expresion | CTE_STRING) )* RPAREN SEMICOLON;

condition     : IF LPAREN expresion RPAREN body (ELSE body)? ;

cycle         : WHILE LPAREN expresion RPAREN DO body ;

expresion     : exp (relop exp)? ;

relop         : EQ
              | NEQ
              | LT
              | LEQ
              | GT
              | GEQ ;

exp           : termino ((PLUS | MINUS) termino)* ;

termino       : factor ((MULT | DIV) factor)* ;

factor        : LPAREN expresion RPAREN
              | CTE_INT
              | CTE_FLOAT
              | ID
              | f_call ;

f_call        : ID LPAREN (expresion (COMMA expresion)*)? RPAREN ;
```


Organización de los Formatos

- Las expresiones regulares para el léxico fueron definidas en la sección de Lexer Rules del archivo .g4.
- Las reglas gramaticales fueron estructuradas en la sección de Parser Rules del mismo archivo.
- Se utilizó una combinación de la notación BNF y EBNF simplificada (propia de ANTLR), que permite expresar repeticiones, opcionales y alternativas de manera compacta.

Caso	Descripción	Entrada de prueba	Resultado
TC1	Programa mínimo válido	<pre>program hola; main { } end</pre>	Aceptado
TC2	Declaración de variables múltiples	<pre>program varTest; var int x, y, z; main { x = 1; y = 2; z = x + y; print(z); } end</pre>	Aceptado
TC3	Función sin parámetros	<pre>program funcSinParams; int suma() : var int a; { a = 1; print(a); }; main { print(1); } end</pre>	Aceptado
TC4	Función con parámetros	<pre>program funcParams; int mult(int x, float y) : var float res; { res = x * y; print(res); }; main { print(2); } end</pre>	Aceptado

TC5	Uso de while	<pre> program ciclo; var int i; main { i = 3; while (i > 0) do { print(i); i = i - 1; } } end </pre>	Aceptado
TC6	Condicional if-else	<pre> program decision; var int a; main { a = 5; if (a < 10) { print("Menor que 10"); } else { print("Mayor o igual que 10"); } } end </pre>	Aceptado
TC7	Error léxico	<pre> program 123Invalid; main { } end </pre>	Error (identificador inválido 123Invalid)
TC8	Error sintáctico (falta ; en asignación)	<pre> program errorSintaxis; var int x; main { x = 10 print(x); } end </pre>	Error (falta punto y coma después de x = 10)
TC9	Llamada a función	<pre> program llamado; int cuadrado(int x) : var int r; { r = x * x; print(r); }; main { cuadrado(5); } end </pre>	Aceptado

TC10	Expresión compuesta	<pre> program expr; var float resultado; main { resultado = (3 + 5) * 2.5; print(resultado); } end </pre>	Aceptado
-------------	------------------------	---	----------

```

badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc1.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc2.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc3.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc4.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc5.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc6.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc7.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc8.bd
line 1:8 extraneous input '123' expecting ID
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc9.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc10.bd
badito@Gabriels-MacBook-Pro Babyduck %

```

Tabla de Consideraciones Semánticas

Aritmetica

Operando Izquierdo	Operando Derecho	+	-	*	/
int	int	int	int	int	int
int	float	float	float	float	float
float	int	float	float	float	float
float	float	float	float	float	float

Relaciones

Operando Izquierdo	Operando Derecho	==	!=	<	<=	>	>=
int	int	bool	bool	bool	bool	bool	bool
int	float	error	error	error	error	error	error
float	int	error	error	error	error	error	error
float	float	bool	bool	bool	bool	bool	bool

Directorio de Funciones

El Directorio de Funciones es una estructura de datos diseñada para registrar toda la información relevante de cada función declarada en el programa BabyDuck. Está implementado mediante un diccionario (HashMap) cuya clave es el nombre de la función, y cuyo valor es una instancia de una clase FunctionInfo. Esta clase almacena el tipo de retorno de la función, la lista de tipos de parámetros y una tabla de variables locales asociada a esa función. Además, el directorio mantiene un campo llamado `currentFunction` que indica cuál es la función activa durante el análisis, lo cual permite asignar variables o parámetros en el contexto correcto.

Esta estructura permite realizar validaciones importantes como:

- Evitar la declaración duplicada de funciones.
- Registrar y acceder al tipo de retorno de cada función.
- Validar la existencia y la cantidad de parámetros durante llamadas a funciones.
- Asociar una tabla de variables para mantener el alcance léxico correctamente definido.

El Directorio de Funciones también contempla una función especial llamada "global" que representa el contexto del programa principal fuera de cualquier función. Las variables globales se almacenan en esta entrada para ser accesibles desde cualquier parte del programa.

Tabla de Variables

La Tabla de Variables es una estructura auxiliar utilizada dentro de cada función del Directorio de Funciones. Está implementada también como un diccionario (HashMap), donde cada clave corresponde al nombre de una variable, y su valor es el tipo de dato asociado a dicha variable (int, float o string). Cada función contiene su propia instancia de esta tabla, lo que permite mantener un ámbito léxico separado entre funciones.

Durante el análisis semántico, cada vez que se declara una variable, se registra en la tabla correspondiente a la función activa. Esta estructura permite realizar validaciones clave como:

- Verificar que una variable no haya sido declarada más de una vez en el mismo ámbito.
- Consultar el tipo de una variable para validar asignaciones y expresiones.
- Distinguir entre variables locales y globales según el contexto actual.

La separación de variables por función permite un control riguroso del alcance y facilita la implementación de reglas semánticas relacionadas con la visibilidad y el uso correcto de identificadores en el lenguaje BabyDuck.

Puntos neurálgicos

1. Inicio del programa
 - Se crea el contexto "global" en el Directorio de Funciones.
 - No requiere validaciones en esta etapa.
2. Declaración de funciones
 - Se registra el nombre y tipo de retorno.
 - Se valida que la función no esté previamente declarada.
3. Registro de parámetros
 - Se agregan a la lista de parámetros y a la tabla de variables local.
 - Se valida que no haya nombres duplicados entre parámetros o con variables locales.
4. Declaración de variables
 - Se registran en la tabla de variables del contexto activo.
 - Se valida que no se repita el nombre dentro del mismo ámbito.
5. Asignaciones
 - Se verifica que la variable esté declarada antes de usarse.
 - Se puede extender para validar compatibilidad de tipos.
6. Uso de identificadores en expresiones
 - Se valida que todas las variables usadas estén declaradas.
 - Se puede obtener su tipo para verificar operaciones posteriores.
7. Evaluación de expresiones
 - Se determina el tipo resultante de operaciones.
 - Se puede aplicar un cubo semántico para validar operadores entre tipos.
8. Gestión del contexto
 - Se mantiene el valor de currentFunction para saber dónde insertar o buscar variables.
 - Asegura que cada declaración o referencia se haga en el ámbito correcto.

Estructuras Pilas y Fila de Cuádruplos

Pilas

1. Pila de Operandos

- Nombre: operandStack
- Tipo: Stack<String>
- Propósito: almacenar secuencialmente los operandos (identificadores, literales o temporales) conforme se recorren expresiones.
- Operaciones clave:
 - pushOperand(valor, tipo): apila un nuevo operando junto con su tipo.
 - popLastOperand(): desapila y retorna el operando más reciente.
 - peekLastOperand(): consulta sin desapilar el operando superior, útil para encadenar operaciones.

2. Pila de Operadores

- Nombre: operatorStack
- Tipo: Stack<String>
- Propósito: guardar los operadores (+, -, *, /, ==, etc.) en el orden en que aparecen, hasta que sea momento de generar el cuádruplo correspondiente.
- Operaciones clave:
 - pushOperator(op): apila el operador op.
 - Al generar el cuádruplo, se hace operatorStack.pop() para obtener el operador que corresponde al par de operandos más reciente.

3. Pila de Tipos

- Nombre: typeStack
- Tipo: Stack<String>
- Propósito: llevar en paralelo el tipo de dato de cada operando apilado (int, float, string o "temp" para resultados intermedios).
- Operaciones clave:
 - Cada vez que haces pushOperand, simultáneamente haces typeStack.push(tipo).
 - Al popLastOperand() corresponde también typeStack.pop() para mantener sincronía.

Filas

1. Fila de Cuádruplos

- Nombre: quadruples
- Tipo: List<Quadruple> (se comporta como fila FIFO para impresión final)
- Propósito: almacenar, en el orden en que se generan, todos los cuádruplos del programa. Cada cuádruplo tiene la forma
- Operaciones clave:
 - generateExpressionQuadruple(): construye un cuádruplo binario, lo añade al final de quadruples y apila el temporal resultado.
 - generateAssignment(var): añade un cuádruplo de asignación.
 - generatePrint(): añade un cuádruplo para la instrucción print.
 - displayQuadruples(): recorre la lista en orden y la imprime en consola al terminar el análisis.

Diagrama de flujo de un lenguaje de programación que define la estructura de un programa, declaraciones de variables, funciones, parámetros, expresiones, factores, condiciones y ciclos.

Programa: El flujo comienza en el nodo 1, pasando por **Pogram**, **Id**, y un punto y coma (;). Luego, se bifurca para **VARS** o **FUNCS**, que ambos convergen en **main**, seguido de **Body** y finalmente **end**.

<VARS>: Comienza con **var** (nodo 2), seguido de **var_decl** y un punto y coma (;). El flujo para **<var_decl>** pasa por **TYPE** y **id** (con un punto y coma (;) entre ellos), regresando a **TYPE** para un bucle.

<FUNCS>: Comienza con **void** (nodo 3), seguido de **id** (nodo 4), un paréntesis de apertura ((), **parametros**, un paréntesis de cierre (), un corchete de apertura [], **VARS** (nodo 4), **Body**, un corchete de cierre] y un punto y coma (;). El flujo para **<parametros>** pasa por **TYPE** y **id** (nodo 5) con un punto y coma (;) entre ellos, regresando a **TYPE** para un bucle.

<assign>: Comienza con **id** (nodo 6), seguido de **=** (nodo 7), **EXPRESSION** (nodo 8) y un punto y coma (;).

<expresion>: Comienza con **EXP**. El flujo puede ir directamente a **<exp>** o pasar por operadores **>**, **<** o **!=** antes de ir a **EXP**.

<exp>: Comienza con **TERMINO**. El flujo puede ir directamente a **<termino>** o pasar por operadores **+** o **-** (nodo 9) antes de ir a **TERMINO**.

<termino>: Comienza con **FACTOR**. El flujo puede ir directamente a **<factor>** o pasar por operadores ***** o **/** (nodo 10) antes de ir a **FACTOR**.

<factor>: Comienza con un paréntesis de apertura ((), seguido de **EXPRESSION** y un paréntesis de cierre (). El flujo puede ir directamente a **<expresion>** o pasar por operadores **+** o **-** antes de ir a **EXPRESSION**. También puede ir a **id** (nodo 11) o **CTE** antes de ir a **EXPRESSION**.

<print>: Comienza con **print** (nodo 12), seguido de un paréntesis de apertura ((), **TYPE** y un paréntesis de cierre () (nodo 13). El flujo puede ir directamente a **<expresion>** o pasar por **cte.string** antes de ir a **TYPE**.

<condition>: Comienza con **if** (nodo 14), seguido de un paréntesis de apertura ((), **EXPRESSION**, un paréntesis de cierre (), **CUERPO** (nodo 15), un punto y coma (;). Si hay un **else**, el flujo va a **CUERPO** antes de ir al punto y coma (;).

<cycle>: Comienza con **while** (nodo 16), seguido de un paréntesis de apertura ((), **EXPRESSION**, un paréntesis de cierre (), **do**, **CUERPO** y un punto y coma (;).

1. visitPrograma: crea la entrada “global” en el Directorio de Funciones y fija el contexto activo.
2. visitVar_decl: por cada ID apila la variable en la tabla de variables del contexto actual.
3. visitFuncion: addFunction(nombre, tipo) y setCurrentFunction(nombre).
4. Tras procesar parámetros y vars, setCurrentFunction("global") al cerrar la función.
5. visitParametros: addParam(tipo) y addVariable(ID, tipo) para registrar parámetros como variables locales.
6. visitAssign valida que la variable exista.

7. visitExpresion y visitExp / visitFactor apilan operandos y generan cuádruplos aritméticos si procede.
8. visitAssign extrae el resultado de la pila y llama a generateAssignment().
9. visitExp detecta operadores +/-, extrae los dos últimos operandos, y genera un cuádruplo binario.
10. visitTermino hace lo mismo con * y /.
11. visitFactor apila cada identificador o literal; al reentrar por subexpresión, delega y apila su resultado.
12. visitPrint invoca quadGen.pushOperand(expr, tipo) para el argumento.
13. visitPrint luego llama a generatePrint() para emitir (print, _, _, expr).
14. visitCondition genera el cuádruplo relacional con generateExpressionQuadruple() y prepara saltos.
15. Al encontrar else, ajusta el salto falso con quadGen (puede representarse como generateGoto).
16. visitCycle gestiona la condición y los saltos de inicio/fin de bucle.

Diagrama de flujo de un compilador para un lenguaje de programación. El diagrama muestra la estructura de un programa y la derivación de sus componentes sintácticos.

Programa: El flujo comienza con el símbolo `<Programa>`, que deriva en el número 1, luego en el símbolo `Pogram` (con una 'P' mayúscula), luego en `id`, luego en el símbolo `:`. Desde aquí, el flujo se ramifica para derivar `VARS` y `FUNCS`, que luego convergen en el símbolo `main`, luego en `Body`, y finalmente en el símbolo `end`.

VARS: El flujo comienza con el símbolo `<VARS>`, que deriva en el símbolo `var` (con el número 2), luego en `var_decl` (con el número 17), luego en el símbolo `:`. Desde aquí, el flujo se ramifica para derivar `TYPE` y `id`, que luego convergen en el símbolo `:`.

FUNCS: El flujo comienza con el símbolo `<FUNCS>`, que deriva en el símbolo `void` (con el número 3), luego en `id` (con el número 4), luego en el símbolo `(`, luego en `parametros`, luego en el símbolo `)`. Desde aquí, el flujo se ramifica para derivar `[` y `Body`, que luego convergen en el símbolo `]`, luego en el símbolo `:`.

parametros: El flujo comienza con el símbolo `<parametros>`, que deriva en `TYPE` (con el número 5), luego en el símbolo `:` (con el número 17), luego en `id` (con el número 17). Desde aquí, el flujo se ramifica para derivar `,` y `id`, que luego convergen en el símbolo `,`.

assign: El flujo comienza con el símbolo `<assign>`, que deriva en `id` (con el número 6), luego en el símbolo `=` (con el número 7), luego en `EXPRESION` (con el número 8), luego en el símbolo `:` (con el número 21).

expression: El flujo comienza con el símbolo `<expression>`, que deriva en `EXP`. Desde aquí, el flujo se ramifica para derivar `>`, `<`, `!=` y `EXP`, que luego convergen en el símbolo `>`.

exp: El flujo comienza con el símbolo `<exp>`, que deriva en `TERMINO`. Desde aquí, el flujo se ramifica para derivar `+`, `-` y `TERMINO`, que luego convergen en el símbolo `+`.

termino: El flujo comienza con el símbolo `<termino>`, que deriva en `FACTOR`. Desde aquí, el flujo se ramifica para derivar `*`, `/` y `FACTOR`, que luego convergen en el símbolo `*`.

factor: El flujo comienza con el símbolo `<factor>`, que deriva en el símbolo `(`. Desde aquí, el flujo se ramifica para derivar `EXPRESION` y `)`, que luego convergen en el símbolo `(`. Desde aquí, el flujo se ramifica para derivar `+`, `-` y `id`, que luego convergen en el símbolo `+`. Desde aquí, el flujo se ramifica para derivar `CTE` y `id`, que luego convergen en el símbolo `CTE`.

print: El flujo comienza con el símbolo `<print>`, que deriva en el símbolo `print` (con el número 12), luego en el símbolo `(` (con el número 20). Desde aquí, el flujo se ramifica para derivar `,` y `TYPE`, que luego convergen en el símbolo `,`. Desde aquí, el flujo se ramifica para derivar `cte.string` y `id`, que luego convergen en el símbolo `cte.string`. Desde aquí, el flujo se ramifica para derivar `)` y `:`, que luego convergen en el símbolo `)`.

condition: El flujo comienza con el símbolo `<condition>`, que deriva en el símbolo `if` (con el número 14), luego en el símbolo `(`. Desde aquí, el flujo se ramifica para derivar `EXPRESION` y `)`, que luego convergen en el símbolo `(`. Desde aquí, el flujo se ramifica para derivar `CUERPO` y `else`, que luego convergen en el símbolo `CUERPO`. Desde aquí, el flujo se ramifica para derivar `CUERPO` y `:`, que luego convergen en el símbolo `CUERPO`.

cycle: El flujo comienza con el símbolo `<cycle>`, que deriva en el símbolo `while` (con el número 16), luego en el símbolo `(`. Desde aquí, el flujo se ramifica para derivar `EXPRESION` y `)`, que luego convergen en el símbolo `(`. Desde aquí, el flujo se ramifica para derivar `do` y `CUERPO`, que luego convergen en el símbolo `do`. Desde aquí, el flujo se ramifica para derivar `CUERPO` y `:`, que luego convergen en el símbolo `CUERPO`.

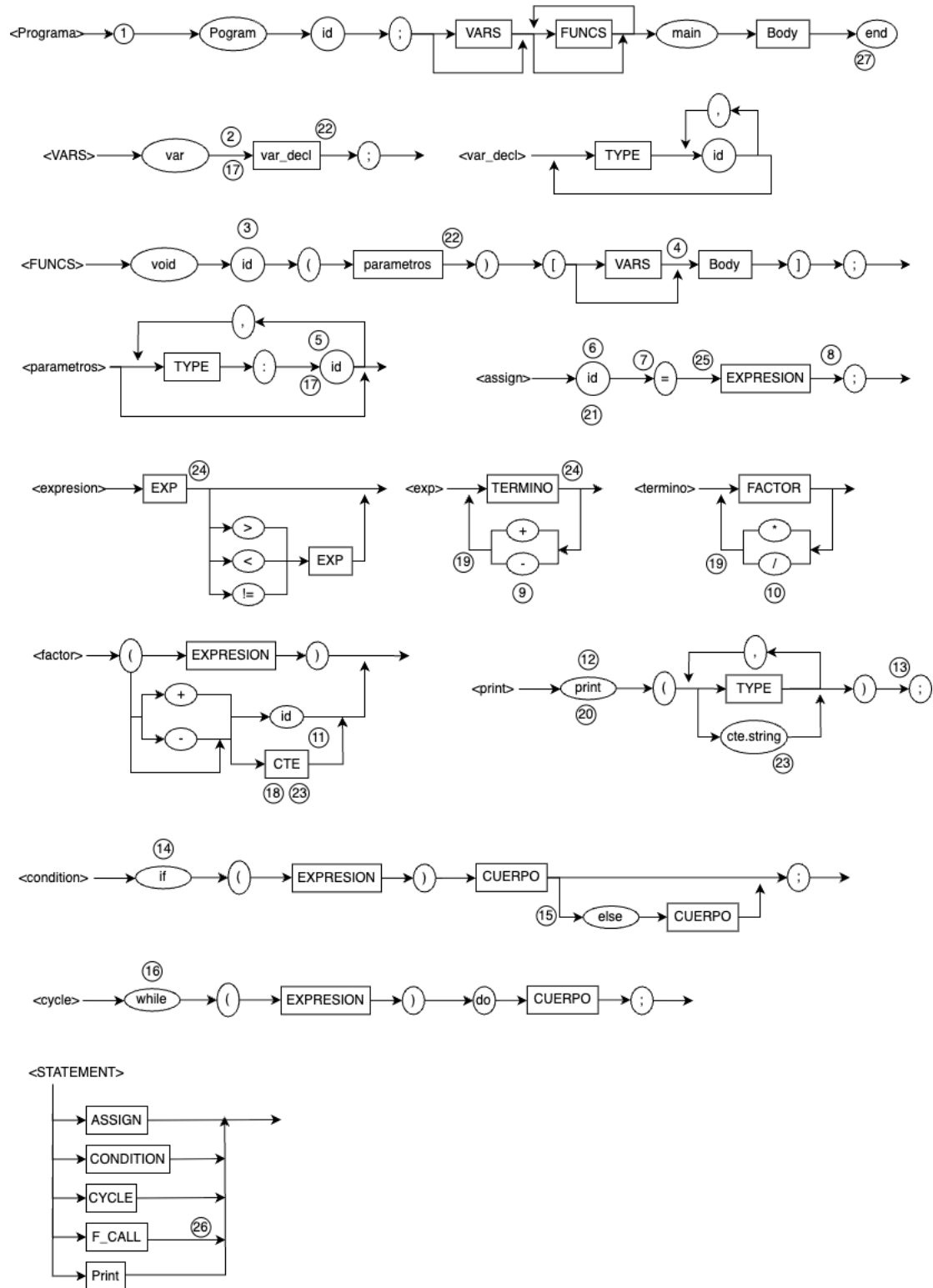
17. `visitVar_decl / visitParametros`: invoca `memory.allocateGlobalVariable(name, type)` o `memory.allocateLocalVariable(func, name, type)` para reservar y registrar la dirección virtual de cada variable declarada.
18. `visitFactor (constante)`: llama a `memory.getConstantAddress(literal, type)` para obtener (o asignar la primera vez) la dirección virtual correspondiente al literal y lo apila como operando.
19. `generateExpressionQuadruple()`: tras extraer los operandos y el operador, invoca `memory.newTemporal(resultType)` para reservar una dirección de temporal, genera el cuádruplo con esa dirección y apila el resultado.

20. visitPrint: obtiene la dirección de la variable con `memory.getVariableAddress(func, id)`, la apila y luego llama a `quadGen.generatePrint()` para emitir el cuádruplo (`print, _, _, dirección`).
21. visitAssign: recupera la dirección de la expresión con `quadGen.popLastOperand()`, obtiene la dirección de la variable destino con `memory.getVariableAddress(func, id)` y finalmente invoca `quadGen.generateAssignment(destAddr)` para producir el cuádruplo de asignación.

Distribución de Direcciones Virtuales

Segmento	Base	Rango	Uso
Global int	1000	1000 – 1999	Variables globales de tipo int
Global float	2000	2000 – 2999	Variables globales de tipo float
Global string	3000	3000 – 3999	Variables globales de tipo string
Local int	5000	5000 – 5999	Variables locales de tipo int
Local float	6000	6000 – 6999	Variables locales de tipo float
Local string	7000	7000 – 7999	Variables locales de tipo string
Temporal int	9000	9000 – 9999	Temporales de tipo int
Temporal float	10000	10000 – 10999	Temporales de tipo float
Temporal string	11000	11000 – 11999	Temporales de tipo string
Constante int	13000	13000 – 13999	Literales enteras (CTE_INT)
Constante float	14000	14000 – 14999	Literales flotantes (CTE_FLOAT)
Constante string	15000	15000 – 15999	Literales de cadena (CTE_STRING)

Diagrama de Lenguaje



22. En `visitVar_decl` y `visitParametros` se invoca a `memory.allocateGlobalVariable` o `memory.allocateLocalVariable` para reservar la dirección virtual de cada variable o parámetro tras su registro semántico.
23. En `visitFactor`, al encontrar `CTE_INT`, `CTE_FLOAT` o `CTE_STRING` se llama a `memory.getConstantAddress(literal, tipo)` y ese valor numérico se apila como operando.
24. Dentro de `generateExpressionQuadruple` se ejecuta `memory.newTemporal(tipoResultado)` para obtener la dirección del temporal donde irá el resultado antes de emitir el cuádruplo.
25. En `visitAssign`, tras evaluar la expresión, se sustituye el nombre de la variable por su dirección obtenida con `memory.getVariableAddress(función, nombreVar)` en el campo `result` del cuádruplo “=”.
26. En `visitF_call` se generan los cuádruplos ERA (`quadGen.generateEra`), PARAM por cada argumento (`quadGen.generateParam`) y GOSUB (`quadGen.generateGoSub`), se cambia el contexto a la función, se visita su cuerpo para generar sus cuádruplos y luego se restaura el contexto global.
27. En `Main.java`, después de `displayOnProgramEnd` se imprime “— Resultados de la Máquina Virtual —”, se instancia `VirtualMachine` y se llama a `vm.run()`, finalizando con “— Fin de ejecución VM —” para delimitar el bloque de ejecución.

Estructuras de Datos para Máquina Virtual

La Máquina Virtual de BabyDuck utiliza las siguientes estructuras de datos clave para gestionar memoria y control de ejecución:

1. Lista de cuádruplos (`List<Quadruple> quads`)

Al arrancar la VM, recibe la lista completa de cuádruplos generados en tiempo de compilación. Cada entrada representa una instrucción atómica (operaciones aritméticas, saltos, llamadas, impresión, etc.). La VM avanza un “Instruction Pointer” (IP) sobre esta lista para ejecutar secuencialmente cada cuádruplo.

2. Memoria global (`Map<Integer, Object> globalMem`)

Un `HashMap` de direcciones a valores que almacena todas las variables globales del programa. Las claves son enteros que representan direcciones virtuales (por ejemplo, 1000–4999 para `int`, 2000–2999 para `float`), y los valores son los objetos Java (`Integer`, `Float`, etc.) que contienen el dato real.

3. Pila de contextos locales (`Deque<Map<Integer, Object>> localStack`)

Cada función, al invocarse (cuádruplo ERA), empuja un nuevo `HashMap` vacío que representará su propio “frame” de variables locales. Cuando se ejecuta un PARAM, el

valor se escribe en el mapa superior de la pila; al finalizar la función, ese frame se desapila, devolviendo el control al contexto anterior.

4. Memoria de temporales (Map<Integer,Object> tempMem)

Un HashMap separado para almacenar el resultado intermedio de expresiones (direcciones 9000–10999, según tipo). Cada vez que se genera un cuádruplo aritmético o relacional, su resultado se almacena en una nueva dirección de temporal y permanece disponible hasta consumirse como operando en instrucciones posteriores.

5. Memoria de constantes (Map<Integer,Object> constMem)

Antes de entrar en el bucle principal, la VM recorre el mapa de literales (memoryManager.getConstantsMap()) y carga cada constante (enteros, flotantes, cadenas) en este HashMap, mapeando dirección (13000–15999) a valor Java. Así, cuando el programa usa un literal, la VM lo recupera directamente de constMem.

6. Instruction Pointer (int IP)

Un simple entero que indica la posición actual en la lista de cuádruplos. Se incrementa tras cada instrucción, o se modifica para realizar saltos condicionales (GOTO), incondicionales (GOTO) o llamadas de función (GOSUB).